

Instituto Tecnológico de San Luis Potosí

Ingeniería en Sistemas Computacionales

Lenguajes de Interfaz 11:00-12:00 Tareas Unidad 4

Alumna:

Venegas Ordaz Fátima Vianney

Lugar y Fecha:

Soledad de Graciano Sánchez, San Luis Potosí, 11 de abril de 2026

Índice

Introducción.	4
Programación para dispositivos de entrada.....	5
Lápiz óptico.	5
Lector de código de barras.....	6
Lector de adquisición de datos.....	7
Scanner.	8
Ratón.....	9
Tabletas digitalizadas.	10
Programación para dispositivos de entrada (Scanner y teclado).	11
Teclado.	11
Scanner.	14
Programación para dispositivos de salida.....	16
Impresoras.	16
Scanner.	17
Monitor LCD.	17
Cámaras de video.	19
Bocinas de Audio.	20
Programación de dispositivos de salida (Impresoras y Cámaras de video)	21
Impresoras.....	21
Cámaras de video.....	23
Herramientas para la programación de dispositivos y su código.	25
STM32CubeIDE.	25
SparkFun RedBoard Artemis.....	26

MPLAB [®] X Entorno de desarrollo integrado (IDE).....	28
Conclusión.....	31
Referencias.....	32

Introducción.

El desarrollo de sistemas computacionales requiere comprender tanto los dispositivos de entrada como los de salida, así como las herramientas que permiten programarlos de manera eficiente. Los dispositivos de entrada, como el lápiz óptico, el lector de código de barras o el ratón, convierten señales físicas en datos digitales que el software interpreta. Los dispositivos de salida, como impresoras, monitores LCD o bocinas de audio, transforman la información procesada en resultados perceptibles para el usuario. Finalmente, las herramientas de programación más allá de Arduino, como STM32CubeIDE, SparkFun Artemis y MPLAB X IDE, ofrecen entornos profesionales que permiten un mayor control sobre el hardware y la optimización de proyectos embebidos. Este conjunto de investigaciones busca mostrar cómo la programación conecta la lógica del software con la realidad física, garantizando la interacción completa entre usuario, máquina y entorno.

Programación para dispositivos de entrada.

Lápiz óptico.

Un lápiz óptico es una herramienta que facilita la interacción con dispositivos de pantalla táctil, como lo son las tablets, los smartphones y algunas computadoras portátiles. (Lenovo México, s.f.)

- Los dispositivos táctiles suelen usar tecnologías capacitivas o resistivas. La mayoría de los lápices ópticos diseñados para pantallas capacitivas simulan las propiedades eléctricas de tus dedos, permitiendo una interacción precisa. Algunos modelos avanzados integran funcionalidades adicionales como sensibilidad a la presión o rechazo de palma, lo que mejora la experiencia en aplicaciones creativas. (Hackaday, 2015)

Hoy en día el lápiz óptico se programa capturando pointers events “eventos de puntero” a través de APIs de interfaz gráfica los cuales nos permiten identificar tanto las coordenadas como la presión ejercida y el tipo de dispositivo:

A continuación, se muestra el código utilizado para la programación de esta herramienta en JavaScript.

Código.

```
const areaDibujo = document.getElementById("lienzo");

// Escuchar eventos de entrada del puntero
areaDibujo.addEventListener("pointerdown", function(evento) {

    // Verificar si el dispositivo de entrada es un lápiz óptico (pen)
    if (evento.pointerType === "pen") {

        console.log("Coordenada X: " + evento.clientX);

        console.log("Coordenada Y: " + evento.clientY);

        console.log("Nivel de presión: " + evento.pressure);
```

}

}); (MDN Web Docs, 2024)

Lector de código de barras.

El lector de código de barras o escáner es un dispositivo electrónico que utiliza un láser incorporado para leer un código de barras y emitir la información (el número) que representa el código, no la imagen.

El láser del lector (la fuente de luz) empieza a leer el código de barras en un espacio blanco (la zona fija, antes de la primera barra), continúa pasando hasta la última línea y finaliza en el espacio blanco que sigue a ésta.

Los lectores de códigos de barras con tecnología láser están compuestos por:

1. **Un diodo emisor de luz (LED) láser:** emite un rayo que es disparado y rebota en un espejo (espejo doble).
2. **Espejo doble:** este espejo se proyecta a su vez sobre otro espejo (espejo móvil) y termina siendo disparado hacia afuera. Cuando estos scanners son activados, sale de ellos una franja de color rojo.
3. **Electroimán:** aunque el rayo es sólo un punto, el segundo espejo, que es el que lo rebota hacia afuera, ejecuta vibraciones muy veloces provocadas por un electroimán que tiene en la parte posterior. Y, por ello, podemos apreciar una franja roja (múltiples movimientos a gran velocidad del segundo espejo que son imperceptibles para el ojo humano).

Hoy en día la gran mayoría de los lectores de código de barras modernos se conectan vía USB y funcionan bajo el protocolo HID (*Human Interface Device*). Esto significa que actúan como un emulador de teclado.

Una vez que el hardware láser decodifica las barras en un número, el escáner simplemente teclea a gran velocidad esos caracteres en el sistema operativo y envía un salto de línea de manera automática. Por lo tanto, la programación a nivel de

software se simplifica a capturar la entrada estándar del sistema, esperando a que el dispositivo termine de ingresar los datos. (MicroPlanet, 2021)

A continuación, se muestra cómo se recibe esta lectura utilizando Python:

Código.

```
print("Por favor, escanee un código de barras...")
```

La función input() pausa la ejecución hasta que el lector emula el tecleo y el 'Enter'

```
codigo_escaneado = input()
```

```
print("El código ingresado por el dispositivo es: " + codigo_escaneado)
```

(MicroPlanet, 2021)

Lector de adquisición de datos.

Adquisición de datos (abreviado DAQ) se refiere al proceso de medición de un fenómeno eléctrico o físico como el voltaje, la corriente, la temperatura, la presión o el sonido con un ordenador o un dispositivo de registro de datos. El dispositivo utilizado para realizar esta medición se denomina sistema de adquisición de datos, que suele constar de un conjunto de sensores o transductores, circuitos de acondicionamiento de señales y un conversor analógico-digital (ADC) que convierte las señales analógicas de los sensores en valores digitales que pueden ser procesados por un ordenador. A continuación, se presenta un ejemplo del código en Python. (IMC Test & Measurement, s.f.)

Código.

```
import serial
```

```
# Configurar la conexión con el puerto COM donde está el DAQ

# a una velocidad de 9600 baudios

conexion_daq = serial.Serial('COM3', 9600)


# Leer una línea completa de datos enviada por el hardware

datos_recibidos = conexion_daq.readline()

print("Lectura del sensor: ", datos_recibidos.decode('utf-8'))


conexion_daq.close() (Liechti, 2020)
```

Scanner.

Un escáner es un dispositivo que digitaliza documentos, imágenes o incluso objetos, convirtiéndolos en archivos digitales que puedes visualizar, editar y almacenar en tu computadora. Es como una fotocopidora digital que captura contenido y lo guarda en un formato digital.

Los escáneres funcionan utilizando una fuente de luz y un sensor fotosensible para capturar el contenido. Al colocar el documento sobre la superficie de escaneo y comenzar el proceso, la luz ilumina el documento y el sensor convierte la luz reflejada en información digital, creando una copia precisa del documento o imagen.

Para interactuar con un escáner en sistemas como lo son Windows, se utiliza la API WIA (Windows Image Acquisition). (Lenovo México, s.f.)

A continuación, se muestra un ejemplo de código en *c#*.

Código.

```
using WIA;
```



```
// Crear el administrador para buscar escáneres en el equipo

DeviceManager gestor = new DeviceManager();

DeviceInfo infoEscaner = gestor.DeviceInfos[1];


// Conectar con el dispositivo físico

Device escanerConectado = infoEscaner.Connect();

Item configuracionEscaneo = escanerConectado.Items[1];


// Ejecutar el escaneo y transferir la imagen a memoria

ImageFile archivImagen = (ImageFile)configuracionEscaneo.Transfer();
(Microsoft Learn, 2018)
```

Ratón.

Un ratón es un dispositivo utilizado para ingresar datos en una computadora de manera que se pueda controlar interactuando con su interfaz gráfica de usuario. Parece una caja pequeña con dos botones y una rueda que se usa para apuntar, hacer clic y arrastrar elementos en la pantalla. (GeeksforGeeks, 2023)

El ratón se mueve sobre una superficie y su movimiento se refleja en la pantalla. (Lenovo México, s.f.)

La programación en interfaces gráficas se basa en la captura de eventos a través de Listeners. A continuación, un ejemplo de código en Java.

Código.

```
import java.awt.event.*;

import javax.swing.*;
```

```

public class InterfazRaton extends JFrame {

    public InterfazRaton() {

        // Añadir el escuchador de eventos del ratón a la ventana

        this.addMouseListener(new MouseAdapter() {

            @Override

            public void mouseClicked(MouseEvent evento) {

                // Obtener las coordenadas del puntero al hacer clic

                int x = evento.getX();

                int y = evento.getY();

                System.out.println("Se registró un clic en las coordenadas: (" + x +
", " + y + ")");

            }

        });

    }

} (Oracle, 2024)

```

Tabletas digitalizadas.

Una tableta digitalizadora, también conocida como tableta gráfica o digitalizador, es un dispositivo de entrada que permite a los usuarios introducir datos y comandos de manera natural y precisa mediante el uso de un lápiz óptico o un dedo.

Hoy en día, las tabletas digitalizadoras cuentan con pantallas sensibles al tacto y a la presión, permitiendo a los usuarios dibujar, escribir y manipular contenido

digital de manera intuitiva. Estas tabletas se integran con software de diseño, edición de imágenes y animación, ofreciendo una experiencia de usuario más natural y fluida. Ahora un pequeño ejemplo de un código en Python. (Aratecnia, s.f.)

Código.

```
from PyQt5.QtWidgets import QWidget

from PyQt5.QtGui import QTabletEvent

class AreaDeDibujo(QWidget):

    # Método que captura nativamente los eventos de la tableta gráfica

    def tabletEvent(self, evento: QTabletEvent):

        nivel_presion = evento.pressure()

        coordenadas = evento.pos()

        print(f"Dibujando en {coordenadas} con un nivel de presión de {nivel_presion}")

        evento.accept() (Riverbank Computing, 2024)
```

Programación para dispositivos de entrada (Scanner y teclado).

Teclado.

Un teclado es un dispositivo de entrada que te permite escribir letras, números y símbolos en tu laptop, PC u otro dispositivo electrónico.

Cuando presionas una tecla en el teclado, envía una señal eléctrica al procesador de tu computadora, que luego interpreta esa señal y muestra la letra o símbolo correspondiente en la pantalla.

En Java, se utiliza la interfaz `KeyListener`. Esta permite capturar el momento exacto en que una tecla es presionada, liberada o escrita. (Lenovo México, s.f.)

Lógica de programación: El hardware del teclado envía un "scancode" (código de rastreo) al sistema operativo cada vez que se cierra el circuito de una tecla. Java traduce esto en un `KeyEvent`, permitiendo al programador identificar la tecla mediante su código virtual (`VK_CODE`). (Oracle, 2024)

Código de implementación.

```
import java.awt.event.*; // Importamos las herramientas de eventos

import javax.swing.*; // Importamos para la ventana gráfica


public class ControlTeclado extends JFrame {


    public ControlTeclado() {

        // Configuramos la ventana básica

        setTitle("Captura de Hardware: Teclado");

        setSize(400, 200);

        setDefaultCloseOperation(JFrame.EXIT_VALUE);


        // Agregamos el KeyListener: el "oído" del programa para el teclado

        this.addKeyListener(new KeyListener() {

            @Override

            public void keyPressed(KeyEvent e) {
```

```

        // PASO 1: El hardware envía un código numérico (ScanCode)

        // PASO 2: Java lo traduce a un Virtual Key Code (VK)

        int codigo = e.getKeyCode();

        // PASO 3: Identificamos qué tecla física se activó

        System.out.println("Se detectó una señal eléctrica de la tecla: " +
codigo);

        // Ejemplo de lógica: si la tecla es el ENTER

        if (codigo == KeyEvent.VK_ENTER) {

            System.out.println("Acción ejecutada: El usuario confirmó con
ENTER.");

        }

    }

    // Estos métodos son obligatorios por la interfaz, aunque no se usen

    @Override

    public void keyReleased(KeyEvent e) {

        // Se activa cuando el usuario deja de presionar la tecla

    }

    @Override

    public void keyTyped(KeyEvent e) {

```

```

        // Se activa cuando la tecla genera un carácter (letras/números)

    }

});

}

}

```

Scanner.

Un escáner es un dispositivo que digitaliza documentos, imágenes o incluso objetos, convirtiéndolos en archivos digitales que puedes visualizar, editar y almacenar en tu computadora.

Para interactuar con un escáner desde Python en entornos como Windows, se utiliza comúnmente la librería pywin32 para comunicarse con la API WIA (Windows Image Acquisition). (Microsoft Learn, 2018)

Lógica de programación: A diferencia de un teclado, el escáner no envía datos constantes. El programa debe enviar una instrucción de "escaneo" al dispositivo. La API WIA gestiona la comunicación con el controlador (driver), activa la lámpara y el sensor del escáner, y devuelve los datos en forma de un flujo de bits que se guarda como imagen. (Hammond, 2023)

Código de implementación.

```

import win32com.client # Librería para hablar con el sistema operativo
Windows

```

```

def ejecutar_escaneo():

    try:

        # PASO 1: Creamos un objeto para manejar dispositivos de imagen
(WIA)

        # Esto es como abrir la puerta a los controladores del escáner

```

```
manager = win32com.client.Dispatch("WIA.DeviceManager")
```

```
# PASO 2: Seleccionamos el hardware. manager.DeviceInfos(1)
```

```
# apunta al primer escáner físico conectado al puerto USB.
```

```
dispositivo = manager.DeviceInfos(1).Connect()
```

```
print("Hardware conectado exitosamente. Iniciando lámpara y sensor...")
```

```
# PASO 3: Accedemos al 'Item' (el lente/sensor del escáner)
```

```
# y ordenamos la transferencia de datos (el escaneo real)
```

```
item_escaneo = dispositivo.Items(1)
```

```
imagen_digital = item_escaneo.Transfer()
```

```
# PASO 4: Los datos binarios recibidos se guardan como archivo .jpg
```

```
ruta_archivo = "resultado_escaneo.jpg"
```

```
imagen_digital.SaveFile(ruta_archivo)
```

```
print(f"El proceso terminó. Archivo generado: {ruta_archivo}")
```

```
except Exception as e:
```

```
# Si el escáner está apagado o no hay hardware, capturamos el error
```

```
print(f"Error de hardware: {e}")
```

```
if __name__ == "__main__":  
    ejecutar_escaneo()
```

Programación para dispositivos de salida.

Impresoras.

Una impresora es un dispositivo de salida gestionado por un gestor de impresión de servidor y diseñado para imprimir documentos en papel.

Una impresora funciona enviando señales electrónicas desde la computadora a la placa de control de la impresora. La placa de control interpreta estas señales en instrucciones para el cabezal de impresión o el cartucho de tóner. El cabezal de impresión o el cartucho de tóner imprime el documento o la imagen en el papel. (Lenovo México, s.f.)

Para lograr que las impresoras reciban señales electrónicas para controlar el tóner se debe comunicar con el gestor de impresión del sistema operativo, el cual traduce los datos del programa a un lenguaje que la placa de control de la impresora pueda interpretar. Ahora un pequeño ejemplo del código en java. (IBM, s.f.)

Código.

```
import javax.print.*;  
  
// Localizar las impresoras instaladas que recibirán las señales electrónicas  
PrintService[] servicios = PrintServiceLookup.lookupPrintServices(null, null);  
  
for (PrintService service : servicios) {  
    System.out.println("Impresora lista para recibir señales: " +  
service.getName());  
} (Oracle, 2024)
```


Scanner.

Un escáner es un dispositivo que digitaliza documentos, imágenes o incluso objetos, convirtiéndolos en archivos digitales que puedes visualizar, editar y almacenar en tu computadora.

Los escáneres utilizan una fuente de luz y un sensor fotosensible para capturar el contenido. Al colocar el documento sobre la superficie de escaneo y comenzar el proceso, la luz ilumina el documento y el sensor convierte la luz reflejada en información digital, creando una copia precisa del documento o imagen.

Es un dispositivo de entrada, pero si se quiere usar de salida entonces el software debe enviar un comando de activación para que la fuente de luz y el sensor comiencen el barrido. A continuación, un ejemplo en Python. (Lenovo México, s.f.)

Código.

```
import twain

# Iniciar el administrador para enviar la orden de escaneo

sm = twain.SourceManager(0)

# Seleccionar el dispositivo que convertirá la luz en datos

escaner = sm.OpenSource()

# Ordenar al hardware que inicie el proceso de digitalización

escaner.RequestAcquire(0, 0) (TWAIN Working Group, s.f.)
```

Monitor LCD.

El monitor LCD, conocido por sus siglas en inglés como Liquid Crystal Display, corresponde a una pantalla de cristal líquido que tiene multitud de funciones, entre las que destacan la visualización de imágenes tanto fijas, como en movimiento.

Los monitores LCD contienen un microprocesador que se encargará de establecer la posición en la que se encuentran cada uno de los píxeles.

Una pantalla LCD cuenta, generalmente, con 2 placas de vidrio, estando una de ellas iluminada por la parte trasera con una luz intensa procedente de las lámparas CCFL (Cold-Cathode Fluorescent Lamps / Lámparas fluorescentes de cátodo frío). Esto permite el brillo de la pantalla.

Una vez que se determina cuál tiene que ser el píxel coloreado, la celda entiende tres sustancias a partir de las que recibe la corriente.

Una vez que se determina el píxel a colorear, la polarización permite, mediante un color básico (verde, rojo y azul), rellenar aquella celda que es propensa a recibir corriente.

La corriente que le llega a cada píxel determina la saturación para cada color, generando lo que se conoce como gama de colores.

Este proceso se repite cada vez que la imagen que aparece en la pantalla cambia.

La programación de un monitor LCD consiste en enviar datos al microprocesador interno de la pantalla para determinar la saturación y corriente de cada píxel. El software utiliza bibliotecas gráficas para definir estos estados de color (RGB) y polarización que resultarán en la imagen final. Ahora un ejemplo en java. (INS Digital, s.f.)

Código.

```
import javax.swing.*;
```

```
public class SalidaPantalla {
```

```
    public static void main(String[] args) {
```

```
        // El software define los píxeles y colores que el monitor LCD mostrará
```

```

JFrame ventana = new JFrame("Control de Píxeles LCD");

ventana.setSize(400, 300);

JLabel etiqueta = new JLabel("Visualización mediante polarización de
píxeles", SwingConstants.CENTER);

ventana.add(etiqueta);

ventana.setVisible(true);

}

} (Oracle, s.f.)

```

Cámaras de video.

Una videocámara o cámara de video es un dispositivo, generalmente portátil, que permite registrar imágenes y sonidos. Convirtiéndolos en señales eléctricas que pueden ser reproducidos por un aparato determinado. En otras palabras, una cámara de vídeo es un traductor óptico.

El elemento imprescindible para poder capturar la imagen es la luz. Las partículas (fotones) de luz salen de su fuente, rebotan en el sujeto y son capturadas por la lente de la cámara.

Para que la cámara funcione como "traductor óptico", el programa debe gestionar la apertura del obturador y la captura de los fotones convertidos en señales eléctricas. La programación permite acceder a este flujo de señales para procesarlas o almacenarlas. Ahora un ejemplo en Python. (Euroinnova, 2024)

Código.

```

import cv2

# Acceder al hardware que registra imágenes y sonidos

video = cv2.VideoCapture(0)

```

```
while True:

    # Capturar la señal eléctrica traducida de la luz

    ret, frame = video.read()

    cv2.imshow('Registro de Imagen en Tiempo Real', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'): break

video.release() (OpenCV, 2024)
```

Bocinas de Audio.

Un altavoz es un dispositivo que convierte las señales eléctricas en ondas sonoras. Te permite escuchar la salida de audio de varios dispositivos, como computadoras, teléfonos inteligentes, televisores y reproductores de música.

Cuando reproduce audio en un dispositivo, como tu smartphone, el dispositivo envía señales eléctricas al altavoz. Dentro del altavoz, estas señales eléctricas se convierten en vibraciones mecánicas mediante un diafragma o cono. Estas vibraciones crean ondas sonoras que viajan por el aire y llegan a tus oídos, lo que te permite escuchar el sonido.

La programación de audio se encarga de enviar las señales eléctricas precisas que el altavoz convertirá en vibraciones mecánicas. El código define la frecuencia y amplitud de estas señales para que el diafragma genere las ondas sonoras deseadas. A continuación, se muestra un ejemplo en Java. (Lenovo México, s.f.)

Código.

```
import javax.sound.sampled.*;

import java.io.File;

// Preparar el flujo de señales eléctricas para enviar al altavoz

Clip altavoz = AudioSystem.getClip();
```

```

        altavoz.open(AudioSystem.getAudioInputStream(new
File("salida_audio.wav"))));

        // Iniciar el envío de señales para crear vibraciones mecánicas

        altavoz.start(); (Oracle, s.f.)

```

Programación de dispositivos de salida (Impresoras y Cámaras de video)

Impresoras.

Una impresora es un dispositivo de salida gestionado por un gestor de impresión de servidor y diseñado para imprimir documentos en papel.

La impresora es un periférico de salida física. Su programación requiere interactuar con el "Gestor de Impresión" del sistema operativo para enviar datos que el hardware pueda convertir en puntos de tinta o tóner.

Lógica de programación: El software no habla directamente con los motores de la impresora, sino que utiliza una API (como javax.print) para crear un Trabajo de Impresión (Print Job). Este trabajo envía una señal electrónica que la placa de control de la impresora traduce en coordenadas para el cabezal de impresión.

Código de implementación.

```

import javax.print.*; // Librería para servicios de impresión

import java.io.*;

public class Milmpresora {

    public static void main(String[] args) {

        // Primero, le pedimos a Java que busque la impresora que tenemos de
        fábrica

        PrintService milmpresora =
        PrintServiceLookup.lookupDefaultPrintService();

```

```

        if (milImpresora != null) {

            System.out.println("Encontré la impresora: " +
milImpresora.getName());

            // Aquí definimos que lo que vamos a mandar es texto sencillo

            DocFlavor tipoTexto = DocFlavor.INPUT_STREAM.AUTODISCOVER;

            // Creamos el "trabajo", que es como la orden para que la impresora
empiece

            DocPrintJob tarea = milImpresora.createPrintJob();

            // Escribimos lo que queremos que salga en el papel

            String texto = "Hola Profe, esta es una prueba de salida desde Java.";

            InputStream flujoDatos = new
ByteArrayInputStream(texto.getBytes());

            Doc miDoc = new SimpleDoc(flujoDatos, tipoTexto, null);

            try {

                // Aquí es donde el programa manda la señal electrónica a la
impresora

                tarea.print(miDoc, null);

                System.out.println("¡Listo! El texto se envió a la impresora.");

            } catch (PrintException e) {

                System.out.println("Hubo un error al conectar con el hardware.");

```

```
    }  
  }  
}  
}
```

Cámaras de video.

Una videocámara o cámara de video es un dispositivo, generalmente portátil, que permite registrar imágenes y sonidos. Convirtiéndolos en señales eléctricas que pueden ser reproducidos por un aparato determinado. En otras palabras, una cámara de vídeo es un traductor óptico.

Su programación consiste en capturar la señal eléctrica que genera la luz al golpear el sensor y mostrarla como una salida digital en tiempo real.

Lógica de programación: El programa utiliza la librería OpenCV para gestionar el hardware. La lente captura fotones que el sensor CMOS convierte en señales eléctricas. El software "lee" estas señales miles de veces por segundo y las traduce en píxeles que forman el video en la pantalla.

Código de implementación.

```
import cv2 # Usamos esta librería que es la mejor para manejar cámaras  
  
def usar_camara():  
    # Conectamos con la cámara (el 0 significa que es la primera que  
    encuentre)  
  
    # Esto le dice al hardware que se encienda  
  
    mi_video = cv2.VideoCapture(0)
```

```
print("Prendiendo la cámara... Presiona 'q' para salir.")
```

```
# Como el video no se detiene, usamos un ciclo 'while' para capturar todo
```

```
while True:
```

```
    # Aquí le pedimos a la cámara que nos dé la imagen que está viendo  
    ahorita
```

```
    ok, cuadro = mi_video.read()
```

```
    if not ok:
```

```
        print("No se pudo leer la señal de la cámara.")
```

```
        break
```

```
# Esta línea abre una ventanita para mostrarnos el video en la pantalla
```

```
cv2.imshow('Mi Video en Vivo', cuadro)
```

```
# Si el alumno (o usuario) presiona la tecla 'q', el video se detiene
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
# Muy importante: al final hay que "soltar" la cámara para que se apague  
el foquito
```

```
mi_video.release()
```

```
cv2.destroyAllWindows()
```



```
print("Cámara apagada correctamente.")
```

```
if __name__ == "__main__":
```

```
    usar_camara()
```

Herramientas para la programación de dispositivos y su código.

STM32CubeIDE.

STM32CubeIDE es un entorno para editar, compilar y depurar código. Los usuarios suelen utilizarlo junto con otras herramientas del ecosistema STM32Cube, como la herramienta de configuración gráfica STM32CubeMX. También es compatible con diversas herramientas de soluciones verticales, como TouchGFXDesigner, STM32Cube AI Studio y Motor Control Workbench.

Utiliza una arquitectura de capas. Tú programas sobre una capa llamada HAL (Hardware Abstraction Layer), que traduce tus comandos a lenguaje de registros que el chip entiende. (STMicroelectronics, 2024)

Código demostración en C.

```
#include "main.h"
```

```
// Función principal del sistema
```

```
int main(void) {
```

```
    // 1. Inicialización de la capa de abstracción de hardware
```

```
    HAL_Init();
```

```
    // 2. Configuración de los pines (GPIO)
```

// En STM32, los periféricos están apagados por defecto para ahorrar energía

```
__HAL_RCC_GPIOA_CLK_ENABLE(); // Habilitar reloj del puerto A
```

```
GPIO_InitTypeDef GPIO_InitStructure = {0};
```

```
GPIO_InitStructure.Pin = GPIO_PIN_5;
```

```
GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP; // Salida Push-Pull
```

```
GPIO_InitStructure.Pull = GPIO_NOPULL;
```

```
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
```

// 3. Bucle infinito de control del dispositivo

```
while (1) {
```

```
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); // Alternar estado físico del  
pin
```

```
    HAL_Delay(500); // Retraso basado en el SysTick del procesador
```

```
}
```

```
} (STMicroelectronics, 2024)
```

SparkFun RedBoard Artemis.

El [SparkFun Artemis](#) es un módulo increíble. ¡Una gran cantidad de funcionalidades en un espacio reducido de 10x15 mm! Pero lo que realmente lo hace potente es la capacidad de escribir bocetos rápidamente y crear proyectos usando solo código Arduino. (SparkFun Electronics, s.f.)

A diferencia de los procesadores básicos, el Artemis utiliza un núcleo ARM Cortex-M4F con una velocidad de hasta 96MHz. Lo que lo hace especial para la programación de dispositivos es su capacidad de "morder" ráfagas de datos de sensores y procesarlas con IA sin necesidad de conectarse a la nube.

Se programa principalmente en C y C++. También soporta TensorFlow Lite, permitiendo cargar modelos de inteligencia artificial (como reconocimiento de voz o gestos) directamente en el chip.

Este código muestra cómo se inicializa el hardware para trabajar con TensorFlow Lite Micro, algo imposible en un Arduino convencional. (Torrente, 2023)

Código en c++.

```
#include "tensorflow/lite/micro/all_ops_resolver.h"

#include "tensorflow/lite/micro/micro_interpreter.h"

#include "tensorflow/lite/schema/schema_generated.h"
```

```
// Definición del área de memoria para el modelo de IA (Tensor Arena)
```

```
const int kTensorArenaSize = 10 * 1024;
```

```
uint8_t tensor_arena[kTensorArenaSize];
```

```
void setup_artemis_ia() {
```

```
    // 1. Cargar el modelo desde la memoria Flash del dispositivo
```

```
    const tflite::Model* model = tflite::GetModel(g_model_data);
```

```
    // 2. Configurar el intérprete para ejecutar el modelo en el núcleo Cortex-
```

```

static tf::MicroInterpreter interpreter(
    model, resolver, tensor_arena, kTensorArenaSize, error_reporter);

// 3. Asignar memoria física del hardware a los tensores
interpreter.AllocateTensors();
}

void loop_ia() {
    // Ejecutar inferencia de IA localmente en el dispositivo
    interpreter.Invoke();
}

```

MPLAB® X Entorno de desarrollo integrado (IDE)

MPLAB X Integrated Development Environment (IDE) es un programa de software ampliable y altamente configurable que incorpora potentes herramientas para ayudarle a descubrir, configurar, desarrollar, depurar y validar diseños embebidos para la mayoría de nuestros microcontroladores y controladores de señales digitales. MPLAB X IDE funciona a la perfección con el ecosistema de software y herramientas de desarrollo de MPLAB, muchas de las cuales son totalmente gratuitas.

Se utiliza un flujo de **Compilación y Depuración (Debug)**. El programador debe conocer el mapa de memoria del dispositivo.

En esta herramienta, el programador debe configurar los "Configuration Bits" (fusibles del hardware) antes de iniciar, lo cual define cómo se comportará el oscilador físico y los sistemas de protección del chip. (Microchip Technology, 2024)

Código de demostración en C.

```
// Configuración de bits (Fusibles del hardware)

#pragma config FOSC = INTOSCIO // Oscilador interno

#pragma config WDTE = OFF      // Perro guardián desactivado (Watchdog
Timer)

#pragma config PWRTTE = ON     // Temporizador de encendido activado


#include <xc.h>                // Biblioteca principal del compilador XC8

#define _XTAL_FREQ 4000000    // Definir frecuencia de 4MHz para retardos


void main(void) {

    // 1. Configuración de registros de dirección (TRIS)

    // TRISB = 0 significa que todo el Puerto B es SALIDA

    TRISB = 0x00;


    // 2. Inicializar el estado de los pines (LAT)

    LATB = 0x00;


    while(1) {

        // 3. Manipulación directa de los niveles lógicos del hardware

        LATBbits.LATB0 = 1;    // Poner el pin RB0 en estado alto (5V)

        __delay_ms(500);       // Retraso de medio segundo
    }
}
```

```
LATBbits.LATB0 = 0;    // Poner el pin RB0 en estado bajo (0V)
```

```
__delay_ms(500);
```

```
}
```

```
}
```

Conclusión.

El análisis conjunto de dispositivos de entrada, salida y herramientas de programación evidencia que la informática moderna depende de la correcta integración entre hardware y software. Cada periférico requiere un tratamiento lógico específico, y cada entorno de desarrollo ofrece ventajas según la complejidad del proyecto. Comprender estos procesos no solo permite diseñar soluciones tecnológicas más robustas, sino también formar profesionales capaces de enfrentar retos reales en sistemas embebidos y aplicaciones interactivas. En conclusión, dominar la programación de dispositivos y explorar alternativas más allá de Arduino constituye un paso esencial para consolidar competencias en ingeniería en sistemas computacionales.

Referencias.

Aratecnia. (s.f.). ¿Qué es una tableta digitalizadora? Glosario de informática. Recuperado de <https://aratecnia.es/glosario/tableta-digitalizadora/>

GeeksforGeeks. (2023, 22 de junio). What is a Mouse in Computer?. Recuperado de <https://www.geeksforgeeks.org/computer-science-fundamentals/what-is-a-mouse-in-computer/>

Hackaday. (2015, 15 de junio). A revolutionary input device 30 years too late. Recuperado de <https://hackaday.com/2015/06/15/a-revolutionary-input-device-30-years-too-late/>

Hammond, M. (2023). Python for Windows (pywin32) Extensions. Recuperado de <https://github.com/mhammond/pywin32>

IMC Test & Measurement. (s.f.). ¿Qué es la adquisición de datos (DAQ)?. Recuperado de <https://www.imc-tm.es/blog/que-es-la-adquisicion-de-datos-daq>

Lenovo México. (s.f.). ¿Qué es un lápiz óptico? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/lapiz-optico/>

Lenovo México. (s.f.). ¿Qué es el mouse de una computadora? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/mouse-computadora/>

Lenovo México. (s.f.). ¿Qué es un escáner y cómo funciona? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/que-es-un-escaner/>

Lenovo México. (s.f.). ¿Qué es el teclado de una computadora? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/teclado/>

Liechti, C. (2020). pySerial API — pySerial 3.4 documentation. Recuperado de https://pyserial.readthedocs.io/en/latest/pyserial_api.html

MDN Web Docs. (2024). Pointer events: Web APIs. Recuperado de https://developer.mozilla.org/en-US/docs/Web/API/Pointer_events

MicroPlanet. (2021, 14 de enero). Qué es y cómo funciona un lector de código de barras. Recuperado de <https://www.microplanet-psl.com/blog/que-es-y-como-funciona-un-lector-de-codigo-de-barras/>

Microsoft Learn. (2018, 31 de mayo). Windows Image Acquisition (WIA) - Win32 apps. Recuperado de <https://learn.microsoft.com/en-us/previous-versions/windows/desktop/wia/-wia-startpage>

Oracle. (2024). KeyEvent (Java Platform SE 8). Recuperado de <https://docs.oracle.com/javase/8/docs/api/java/awt/event/KeyEvent.html>

Oracle. (2024). MouseListener (Java Platform SE 8). Recuperado de <https://docs.oracle.com/javase/8/docs/api/java/awt/event/MouseListener.html>

Riverbank Computing. (2024). QTabletEvent Class - PyQt5 Reference Guide. Recuperado de <https://www.riverbankcomputing.com/static/Docs/PyQt5/api/qtgui/qtabletevent.html>

Microchip Technology. (2024). MPLAB® X Integrated Development Environment (IDE). Recuperado de <https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide>

SparkFun Electronics. (s.f.). Artemis Development with Arduino: All tutorials. Recuperado de <https://learn.sparkfun.com/tutorials/artemis-development-with-arduino/all>

STMicroelectronics. (2024). STM32CubeIDE: Integrated Development Environment for STM32. Recuperado de <https://www.st.com/en/development-tools/stm32cubeide.html>

Torrente, A. (2023). Alternativas a Arduino: Microcontroladores de alto rendimiento y bajo consumo. Recuperado de <https://descubrearduino.com/alternativas-arduino/>

Euroinnova Business School. (2024). ¿Cómo funciona una cámara de vídeo?. Recuperado de <https://www.euroinnova.com/blog/como-funciona-una-camara-de-video>

IBM. (s.f.). Conceptos de impresora: Gestión de impresión y servidores. Recuperado de <https://www.ibm.com/docs/es/cmofm/9.5.0?topic=concepts-printer>

INS Digital. (s.f.). Monitor LCD (Liquid Crystal Display): Funcionamiento y tecnología. Recuperado de <https://www.ins-digital.com/monitor-lcd-liquid-crystal-display/>

Lenovo México. (s.f.). ¿Cómo funciona una impresora? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/como-funciona-una-impresora/>

Lenovo México. (s.f.). ¿Qué es un altavoz? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/altavoz/>

Lenovo México. (s.f.). ¿Qué es un escáner y cómo funciona? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/que-es-un-escaner/>

OpenCV. (2024). Getting Started with Video: Capture, Display, and Save. Recuperado de https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

Oracle. (2024). Java Print Service User Guide. Recuperado de <https://docs.oracle.com/javase/8/docs/technotes/guides/jps/index.html>

Oracle. (s.f.). Creating a GUI With JFC/Swing: The Java Tutorials. Recuperado de <https://docs.oracle.com/javase/tutorial/uiswing/>

Oracle. (s.f.). Playing Back Audio: The Java Tutorials. Recuperado de <https://docs.oracle.com/javase/tutorial/sound/playing.html>

TWAIN Working Group. (s.f.). TWAIN Standard: Linking Applications and Image Acquisition Devices. Recuperado de <https://twain.org/#/user/home>